Atomic memory ordering: verified notes (Rust and C++)

Checked on October 25, 2026 against the Rust std docs, cppreference, WG21 papers, CPU mapping tables and the loom project. Claims I could not confirm are listed in section 8 and are not part of the notes.

1. The core idea

Two things can make another thread observe your writes in a different order than you wrote them: the compiler (it may reorder when a single thread cannot tell) and the CPU (writes can sit in a store buffer before other cores see them). Orderings are how you forbid that, only where you need it.

2. The five orderings (verified)

Ordering	What it promises	Where it applies
Relaxed	The operation is atomic. No ordering of other memory.	load, store, read-modify-write
Release	When paired with a store: earlier operations become ordered before any Acquire (or stronger) load of this value.	store, read-modify-write
Acquire	When paired with a load: if it reads a Release (or stronger) store, later operations are ordered after that store.	load, read-modify-write
AcqRel	Acquire for the load half, Release for the store half.	read-modify-write only
SeqCst	Acquire/Release/AcqRel plus one total order of all SeqCst operations that every thread agrees on.	everything

Extra rules confirmed in the docs:

- Loads accept Relaxed, Acquire, SeqCst. Stores accept Relaxed, Release, SeqCst. Other values panic (or are flagged by a lint).
- Using Release on a read-modify-write makes its load half Relaxed. Using Acquire makes its store half Relaxed.

- compare_exchange takes a success ordering and a failure ordering. The failure ordering cannot be Release or AcqRel, because a failed compare_exchange does not write.
- fence accepts Acquire, Release, AcqRel, SeqCst and panics on Relaxed.
- Rust's orderings are the same as C++20's.

3. Patterns and the ordering each one needs

Counter or ID: Relaxed

`fetch_add(1, Relaxed)` never loses an increment and never hands out a duplicate. Nothing else depends on when it happens, so nothing stronger is needed.

Publish data with a flag: Release store, Acquire load

Thread A writes the data (Relaxed is fine), then stores the flag with Release. Thread B loads the flag with Acquire. If B sees true, B is guaranteed to see the data.

Spinlock: Acquire to lock, Release to unlock

A spinlock is a lock where a waiting thread loops and retries instead of sleeping. The lock is one atomic flag. A successful lock is an Acquire, unlock is a Release store, and the failure case of the compare_exchange can be Relaxed. The Rust std docs show a spinlock like this, and also a variant that uses a Relaxed compare_exchange followed by fence(Acquire).

Countdown latch or refcount: AcqRel

Each worker adds its result (Relaxed), then calls fetch_sub(1, AcqRel). The Release half publishes the worker's result. The Acquire half lets whoever sees the counter reach zero see all the results.

Store then read the other variable: SeqCst

Thread A stores X then reads Y. Thread B stores Y then reads X. With SeqCst, both reading false is impossible because it would require a loop in the single global order. With only Release/Acquire it is allowed.

Fences: same idea, separated from the atomic

`fence(Release)` before a Relaxed store pairs with `fence(Acquire)` after a Relaxed load (or with an Acquire load, or a Release store). Fences synchronize only through an atomic variable. They cannot order plain non-atomic accesses on their own.

4. Rust and C++ side by side

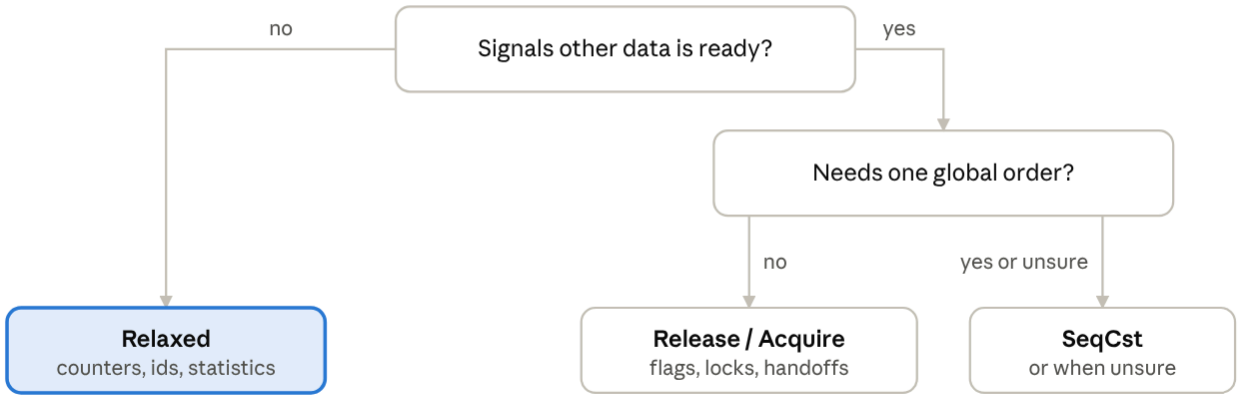
Topic	Rust	C++
Names	Relaxed, Acquire, Release, AcqRel, SeqCst	relaxed, acquire, release, acq_rel, seq_cst
Default	Always pass an ordering	seq_cst if you omit it
consume	Does not exist	Exists, but all current compilers treat it as acquire. Deprecated in C++26 (P3475R2).
Model	Same as C++20	C++20
Mutex lock	not checked	lock() is an acquire operation (cppreference)

5. What the hardware does (verified, with caveats)

- x86: Relaxed, Release and Acquire operations and SeqCst loads compile to a plain MOV. A SeqCst store needs XCHG (or MOV plus MFENCE). So on x86 the extra cost of SeqCst is on stores.
- ARMv8: acquire loads use LDAR, release stores use STLR, and SeqCst uses the same instructions. The cost gap between SeqCst and Release/Acquire is smaller there than on x86.
- The store-buffering outcome (both threads read the old value) really occurs on x86, because a store can sit in the store buffer while the later load goes ahead. This happens even with Release/Acquire, since those compile to plain MOVs.
- ARMv8 STLR/LDAR forbid that outcome in hardware, but the C++ model still allows it. Code must follow the model, not one CPU.
- The mapping table I used describes itself as collected for discussion, not definitive. Treat the exact instructions as typical, not guaranteed for every compiler.

6. Diagrams

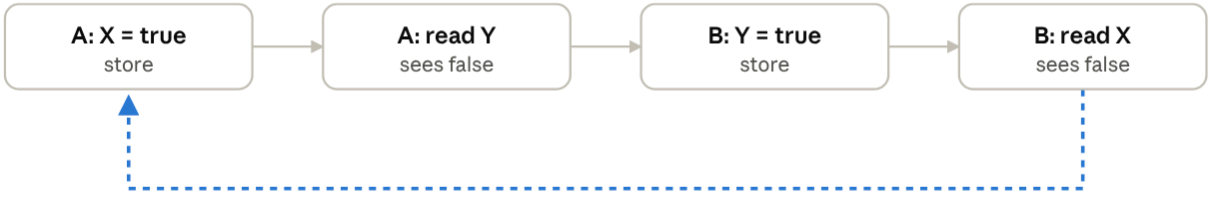
Pick Relaxed unless the value signals that other data is ready



choosing an ordering · 2 questions, 3 answers

Start from Relaxed and move up only when the value tells another thread that data is ready.

Both reads seeing false needs a loop, which no single order can hold



B reads false only if B reads before A stores: the dashed step closes the loop

SeqCst order · 4 operations, 1 impossible loop

SeqCst forbids the loop, so at least one thread sees true. Release/Acquire has no single order and allows both reads to see false.

7. What I got wrong or oversimplified

1. SeqCst cost, ARM versus x86. The pasted notes said SeqCst is more noticeable on ARM than on x86, and I repeated "slightly slower" without checking. The mappings show the opposite direction: on x86 a SeqCst store needs XCHG or MFENCE while a Release store is a plain MOV, whereas ARMv8 uses STLR/LDAR for both. Real cost still depends on the chip. I found no measured numbers.

2. "On x86 the wrong ordering would probably still pass." True for the message-passing pattern, because the hardware keeps stores in order. But the compiler is still free to reorder Relaxed operations, and the store-buffering case fails on x86 even with Release/Acquire.
3. "Release and Acquire only work as a pair on the same variable." Too narrow. Atomic-to-atomic pairing is on one variable, but a fence can stand in on either side, and the Rust docs describe fence-to-fence pairing through any atomic object.
4. "A fence has the same effect as a Release/Acquire operation." Too loose. cppreference says a fence imposes stronger requirements than an acquire or release operation, and the Rust docs call the reordering-barrier picture a simplified model.
5. Test tools. I said loom and ThreadSanitizer work for both Rust and C++. loom is a Rust tool that explores executions under the C11 memory model. ThreadSanitizer works for C++, and for Rust it needs a nightly toolchain with an instrumented std.

8. Discarded or not verified

- "Per-core caches" as a cause of reordering. The mechanism I could confirm is the store buffer. I did not confirm the cache part, so I dropped it.
- Whether Rust's own Mutex gives Acquire/Release internally. I only confirmed it for C++ std::mutex.
- Any claim about how much slower SeqCst is in practice. No source checked.
- Any exact behaviour on a specific CPU model beyond the mappings above.

9. My own test run

I compiled and ran all the example patterns (Rust 1.75, g++ 13 with -std=c++20), 2,000 loops each, and they all passed. That shows the code is valid and the intended outcome holds. It is weak evidence that the orderings are minimal or sufficient, because I did not check the CPU and strong hardware hides ordering bugs. Use loom (Rust) or ThreadSanitizer to test ordering choices properly.

10. Sources

- Rust std::sync::atomic::Ordering: https://doc.rust-lang.org/std/sync/atomic/enum.Ordering.html
- Rust std::sync::atomic::fence: https://doc.rust-lang.org/std/sync/atomic/fn.fence.html
- Rust compare_exchange failure ordering lint: https://doc.rust-lang.org/nightly/nightly-rustc/rustc_lint/types/static.INVALID_ATOMIC_ORDERING.html

- cppreference std::memory_order: https://en.cppreference.com/w/cpp/atomic/memory_order
- P3475R2, defang and deprecate memory_order::consume: https://open-std.org/JTC1/SC22/WG21/docs/papers/2025/p3475r2.pdf
- GCC bug 119062 (consume treated as acquire): https://gcc.gnu.org/bugzilla/show_bug.cgi?id=119062
- C/C++11 mappings to processors (Cambridge): https://www.cl.cam.ac.uk/~pes20/cpp/cpp0xmappings.html
- AArch64 and C11 atomics (DPDK slides): https://www.dpdk.org/wp-content/uploads/sites/23/2019/10/StateofC11Code.pdf
- Store buffering under TSO: https://www.arxiv.org/pdf/0909.4637
- loom: https://github.com/tokio-rs/loom